

✓ Task

Analyze the insurance dataset from "archive.zip" to predict insurance costs using Support Vector Regression (SVR) models with RBF and Polynomial kernels, then compare their performance based on MAE, RMSE, and R^2 scores.

✓ Load and Prepare Data

Subtask:

Load the dataset and preprocess it by encoding categorical variables.

Reasoning: The first instruction is to unzip the `archive.zip` file. I will use the `unzip` command for this.

```
import zipfile
import os

zip_file_path = '/content/archive.zip'
extraction_path = '/content/dataset'

# Create the extraction directory if it doesn't exist
os.makedirs(extraction_path, exist_ok=True)

with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extraction_path)

print(f"Files extracted to: {extraction_path}")

# List extracted files to confirm
print("Extracted files:")
for root, dirs, files in os.walk(extraction_path):
    for file in files:
```

```
print(os.path.join(root, file))
```

```
Files extracted to: /content/dataset  
Extracted files:  
/content/dataset/insurance.csv
```

Reasoning: The next step is to load the extracted CSV file into a pandas DataFrame as per the subtask instructions.

```
import pandas as pd  
  
df = pd.read_csv(os.path.join(extraction_path, 'insurance.csv'))  
  
print("Dataset loaded successfully. Displaying the first 5 rows:")  
print(df.head())
```

```
Dataset loaded successfully. Displaying the first 5 rows:  
   age  sex  bmi  children  smoker  region  charges  
0   19 female  27.900        0    yes southwest  16884.92400  
1   18  male  33.770        1     no  southeast  1725.55230  
2   28  male  33.000        3     no  southeast  4449.46200  
3   33  male  22.705        0     no northwest  21984.47061  
4   32  male  28.880        0     no northwest   3866.85520
```

Reasoning: Now that the data is loaded, I need to identify all categorical columns in the DataFrame as per the subtask instructions. I will use `df.select_dtypes(include='object')` to find them.

```
categorical_cols = df.select_dtypes(include='object').columns  
print("Identified categorical columns:")  
print(categorical_cols.tolist())
```

```
Identified categorical columns:  
['sex', 'smoker', 'region']
```

Reasoning: As per the subtask instructions, I will now apply one-hot encoding to the identified categorical columns using `pd.get_dummies` with `drop_first=True` to handle multicollinearity. Then, I will concatenate the new encoded columns

with the original DataFrame and drop the old categorical columns.

```
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print("DataFrame after one-hot encoding categorical columns and dropping originals. Displaying first 5 rows:")
print(df_encoded.head())
print(f"Shape of DataFrame after encoding: {df_encoded.shape}")
```

DataFrame after one-hot encoding categorical columns and dropping originals. Displaying first 5 rows:

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest \
0	19	27.900	0	16884.92400	False	True	False
1	18	33.770	1	1725.55230	True	False	False
2	28	33.000	3	4449.46200	True	False	False
3	33	22.705	0	21984.47061	True	False	True
4	32	28.880	0	3866.85520	True	False	True

	region_southeast	region_southwest
0	False	True
1	True	False
2	True	False
3	False	False
4	False	False

Shape of DataFrame after encoding: (1338, 9)

✓ Feature Selection

Subtask:

Select the appropriate features for training the SVR models.

Reasoning: As per the instructions, I will define the target variable (y) as the 'charges' column and the feature matrix (X) by dropping 'charges' from (df_encoded).

```
y = df_encoded['charges']
X = df_encoded.drop('charges', axis=1)
```

```
print("Target variable 'y' and feature matrix 'X' defined.")
print(f"Shape of X: {X.shape}")
print(f"Shape of y: {y.shape}")
```

```
Target variable 'y' and feature matrix 'X' defined.
Shape of X: (1338, 8)
Shape of y: (1338,)
```

Reasoning: Following the subtask instructions, I will now split the dataset into training and testing sets using `train_test_split` and then apply feature scaling using `StandardScaler` to prepare the data for model training.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Data split into training and testing sets.")
print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

# Apply feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Features scaled using StandardScaler.")
print("X_train_scaled and X_test_scaled are ready for model training.")
```

```
Data split into training and testing sets.
Shape of X_train: (1070, 8)
Shape of X_test: (268, 8)
Shape of y_train: (1070,)
Shape of y_test: (268,)
Features scaled using StandardScaler.
```

X_train_scaled and X_test_scaled are ready for model training.

✓ Train SVR (RBF Kernel)

Subtask:

Train a Support Vector Regressor model using the RBF kernel.

✓ Reasoning:

To train a Support Vector Regressor (SVR) model with an RBF kernel, I will follow these steps:

1. Import the `SVR` class from `sklearn.svm`.
2. Instantiate an `SVR` model, specifying `kernel='rbf'`. I will use default values for regularization (`C`) and kernel coefficient (`gamma`) initially.
3. Train the model using the `fit` method on the scaled training data, `X_train_scaled` and `y_train`.

Reasoning: As per the instructions and the reasoning provided, I will now implement the code to import `SVR`, instantiate an SVR model with an RBF kernel, and train it using the scaled training data.

```
from sklearn.svm import SVR

# Instantiate SVR model with RBF kernel
svr_rbf = SVR(kernel='rbf')

# Train the model
svr_rbf.fit(X_train_scaled, y_train)

print("SVR model with RBF kernel trained successfully.")
```

SVR model with RBF kernel trained successfully.

✓ Evaluate SVR (RBF Kernel)

Subtask:

Evaluate the RBF kernel SVR model using MAE, RMSE, and R^2 Score.

Reasoning: To evaluate the trained SVR RBF kernel model, I will first make predictions on the scaled test set and then calculate the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score using sklearn's metrics functions.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```
# Make predictions on the scaled test set
y_pred_rbf = svr_rbf.predict(X_test_scaled)
```

```
# Calculate evaluation metrics
mae_rbf = mean_absolute_error(y_test, y_pred_rbf)
mse_rbf = mean_squared_error(y_test, y_pred_rbf)
rmse_rbf = np.sqrt(mse_rbf)
r2_rbf = r2_score(y_test, y_pred_rbf)
```

```
# Print the results
print("SVR (RBF Kernel) Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_rbf:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_rbf:.2f}")
print(f" $R^2$  Score: {r2_rbf:.2f}")
```

```
SVR (RBF Kernel) Model Evaluation:
Mean Absolute Error (MAE): 8612.41
Root Mean Squared Error (RMSE): 12889.10
 $R^2$  Score: -0.07
```

✓ Train SVR (Polynomial Kernel)

Subtask:

Train a Support Vector Regressor model using the Polynomial kernel.

✓ Reasoning:

To train a Support Vector Regressor (SVR) model with a Polynomial kernel, I will follow these steps:

1. Import the `SVR` class from `sklearn.svm`.
2. Instantiate an `SVR` model, specifying `kernel='poly'`.
3. Train the model using the `fit` method on the scaled training data, `X_train_scaled` and `y_train`.

Reasoning: As per the instructions and the reasoning provided, I will now implement the code to import `SVR`, instantiate an SVR model with a Polynomial kernel, and train it using the scaled training data.

```
from sklearn.svm import SVR

# Instantiate SVR model with Polynomial kernel
svr_poly = SVR(kernel='poly')

# Train the model
svr_poly.fit(X_train_scaled, y_train)

print("SVR model with Polynomial kernel trained successfully.")
```

```
SVR model with Polynomial kernel trained successfully.
```

✓ Evaluate SVR (Polynomial Kernel)

Subtask:

Evaluate the Polynomial kernel SVR model using MAE, RMSE, and R^2 Score.

Reasoning: To evaluate the trained SVR Polynomial kernel model, I will first make predictions on the scaled test set and then calculate the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score using sklearn's metrics functions, similar to the RBF kernel evaluation.

Reasoning: As per the instructions and the reasoning provided, I will now implement the code to make predictions using the trained Polynomial SVR model and calculate the evaluation metrics (MAE, RMSE, R^2 Score).

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Make predictions on the scaled test set
y_pred_poly = svr_poly.predict(X_test_scaled)

# Calculate evaluation metrics
mae_poly = mean_absolute_error(y_test, y_pred_poly)
mse_poly = mean_squared_error(y_test, y_pred_poly)
rmse_poly = np.sqrt(mse_poly)
r2_poly = r2_score(y_test, y_pred_poly)

# Print the results
print("SVR (Polynomial Kernel) Model Evaluation:")
print(f"Mean Absolute Error (MAE): {mae_poly:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_poly:.2f}")
print(f" $R^2$  Score: {r2_poly:.2f}")
```

```
SVR (Polynomial Kernel) Model Evaluation:
Mean Absolute Error (MAE): 8607.80
Root Mean Squared Error (RMSE): 12872.96
 $R^2$  Score: -0.07
```


Compare Models and Conclude

Subtask:

Compare the evaluation metrics of both SVR models and conclude which kernel performs best for predicting insurance costs.

Summary:

Q&A

Based on the comparison of the SVR models with RBF and Polynomial kernels, the Polynomial kernel performs marginally better than the RBF kernel. Both models, however, show very poor performance with negative R^2 scores, indicating they perform worse than a simple horizontal line through the mean of the actual values.

Data Analysis Key Findings

- The `archive.zip` file was successfully extracted, and `insurance.csv` was loaded into a DataFrame.
- Categorical columns (`sex`, `smoker`, `region`) were identified and successfully one-hot encoded, resulting in a DataFrame with 9 features.
- The dataset was split into training and testing sets (80% train, 20% test), and features were scaled using `StandardScaler`.
- **SVR model with RBF kernel:**
 - Mean Absolute Error (MAE): 8612.41
 - Root Mean Squared Error (RMSE): 12889.10
 - R^2 Score: -0.07
- **SVR model with Polynomial kernel:**
 - Mean Absolute Error (MAE): 8607.80
 - Root Mean Squared Error (RMSE): 12872.96

- R^2 Score: -0.07
- Both SVR models, using default parameters, exhibited very poor predictive performance as indicated by the negative R^2 scores, which means they perform worse than simply predicting the mean of the target variable. The Polynomial kernel showed slightly lower MAE and RMSE compared to the RBF kernel.

Insights or Next Steps

- The current SVR models with default hyperparameters are not suitable for predicting insurance costs, as indicated by the negative R^2 scores. Significant hyperparameter tuning (e.g., for `C`, `gamma`, `epsilon`, and `degree` for the polynomial kernel) is required to improve their performance.
- Given the poor performance of SVR with default settings, exploring other regression models such as Random Forest Regressor, Gradient Boosting Regressor, or even simpler linear models might be more effective for this dataset, or preprocessing steps should be re-evaluated.